

Skill-4

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano parser.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc parser.c -o parser
himasree@LAPTOP-41COT5AA:~/OSLab$ ./parser
Enter command: ls -l
```

```
Tokens:
ls
-l

Parse Tree:
|-- COMMAND
    |-- ls
    |-- -l
```

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

typedef struct {
    char value[30];
} Token;

typedef struct Node {
    char value[30];
    struct Node *left, *right;
} Node;

/* Tokenize input */
int tokenize(char *s, Token t[]) {
    int n = 0;
    char *p = strtok(s, " \\t\\n|");

    while (p && n < MAX) {
        strcpy(t[n++].value, p);
        p = strtok(NULL, " \\t\\n|");
    }
    return n;
}

/* Create parse tree node */
```

```
Node *newNode(char *s) {
    Node *n = malloc(sizeof(Node));
    strcpy(n->value, s);
    n->left = n->right = NULL;
    return n;
}

/* Display parse tree */
void tree(Node *n, int level) {
    if (!n) return;

    for (int i = 0; i < level; i++)
        printf(" ");

    printf("|-- %s\\n", n->value);

    tree(n->left, level + 1);
    tree(n->right, level + 1);
}

/* Free memory */
void freeTree(Node *n) {
    if (!n) return;

    freeTree(n->left);
    freeTree(n->right);
    free(n);
}

int main() {
```

```

char input[MAX];
Token tokens[MAX];

printf("Enter command: ");
fgets(input, MAX, stdin);

if (strlen(input) <= 1) {
    printf("Error: Empty command\n");
    return 0;
}

int n = tokenize(input, tokens);

printf("\nTokens:\n");
for (int i = 0; i < n; i++)
    printf("%s\n", tokens[i].value);

printf("\nParse Tree:\n");

Node *root = newNode("COMMAND");

if (n > 0)
    root->left = newNode(tokens[0].value);

if (n > 1)
    root->right = newNode(tokens[1].value);

tree(root, 0);

printf("\nParsing successful.\n");

```

```

    freeTree(root);

    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```